

Hybrid Intrusion Detection System (IDS) — Network Module (Snort + Analyzer)

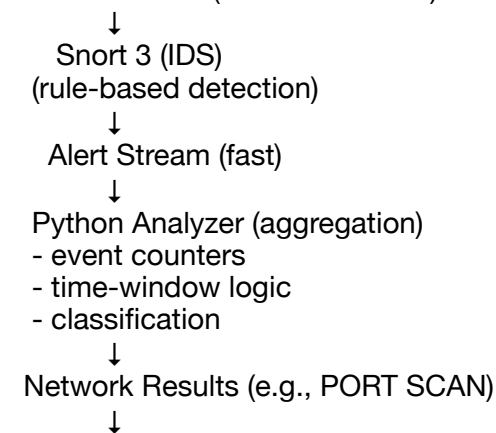
1) Objective

To design and implement a network-layer intrusion detection module for an IoT smart-building environment that:

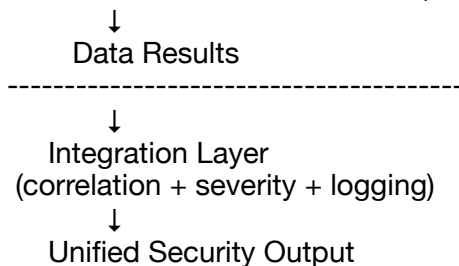
- * Detects pre-attack and in-flight behaviors (recon, scanning, probing)
- * Produces structured alerts for integration with an ML-based anomaly detector
- * Enables hybrid detection by combining network events with sensor-data anomalies

2) System Architecture (End-to-End)

Attacker Traffic (ICMP/TCP/UDP)



IoT Sensor Data → ML Models (LSTM/RF/IF/GB)



3) Environment & Tools

- * OS: Kali Linux (VM)
- * IDS Engine: Snort 3 (Snort++)
- * Interface: eth0 (validated via ip a)
- * Config: /etc/snort/snort.lua (minimal, stable)
- * Rules File: /etc/snort/rules/local.rules
- * Testing: nmap, ping, curl, nslookup
- * Integration: Python 3 (subprocess / log-tail analyzer)

4) Snort Configuration (Stable Minimal Setup)

HOME_NET = '10.0.2.0/24'

```
EXTERNAL_NET = 'any'
include 'snort_defaults.lua'
ips =
{
  variables = default_variables,
  rules = [[
    include /etc/snort/rules/local.rules
  ]]
}
```

Rationale

- * Minimal config avoids module conflicts
- * Guarantees rule loading
- * Suitable for VM-based capture

5) Enhanced Rule Set (IoT-Focused)

A layered rule set was designed to cover multiple stages of an attack:

```
alert icmp any any -> any any (msg:"[IoT] Ping Detected"; sid:1000001; rev:1;)
alert tcp any any -> any any (flags:S; msg:"[IoT] SYN Packet Detected"; sid:1000002; rev:1;)
alert tcp any any -> any 80 (msg:"[IoT] HTTP Access"; sid:1000003; rev:1;)
alert tcp any any -> any 22 (msg:"[IoT] SSH Access Attempt"; sid:1000004; rev:1;)
alert udp any any -> any 53 (msg:"[IoT] DNS Activity"; sid:1000005; rev:1;)
```

6) Detection Coverage (Attack Lifecycle)

Stage	Behavior	Rule	Threat
Reconnaissance		Ping sweep	ICMP Host discovery
Discovery	SYN flood	TCP SYN	Port scanning
Service Access		HTTP traffic	TCP:80 Web probing
Remote Access		SSH attempt	TCP:22 Brute-force entry
Communication		DNS queries	UDP:53 Data exfiltration / malware

7) Snort Operation

Snort runs in IDS mode:

```
sudo snort -i eth0 -A alert_fast -c /etc/snort/snort.lua
```

Output example:

```
[IoT] SYN Packet Detected
```

8) Traffic Simulation (Validation)

- * Ping Sweep

```
ping 10.0.2.15
```

- * Port Scan

```
nmap -sS -p 1-1000 10.0.2.15
```

* HTTP Access

```
curl http://10.0.2.15
```

* DNS Query

```
nslookup google.com
```

9) Python Analyzer (Behavior Layer)

Purpose

Convert packet-level alerts into attack-level decisions.

Core Idea

Detection = Frequency + Pattern + Time Window

Logic

```
if event_counter["SYN"] > threshold:  
    → PORT SCAN DETECTED
```

Example Classification

Pattern Interpretation

Few SYN packets	Normal
Many SYN packets	Port scan
Repeated ICMP	Ping sweep
Frequent HTTP	Service probing

Output

 PORT SCAN DETECTED

 PING SWEEP DETECTED

10) Integration with ML Module

Network Output

```
network_result = ["PORT SCAN DETECTED"]
```

ML Output

```
ml_result = ["INJECTION ATTACK DETECTED"]
```

Combined Output



NETWORK: PORT SCAN DETECTED



DATA: INJECTION ATTACK DETECTED

11) Correlation & Decision Layer

if network_result and ml_result:
→ HIGH CONFIDENCE ATTACK

12) Logging System

All alerts stored:

alerts.log

Format:

[time] NETWORK: PORT SCAN

[time] DATA: INJECTION

13) Key Contributions (Your Work)

- * Implemented Snort 3 IDS setup on Kali VM
 - * Designed multi-stage rule set (ICMP, SYN, HTTP, SSH, DNS)
 - * Resolved:
 - * Interface binding issues (eth0)
 - * Configuration errors (snort.lua)
 - * Rule loading failures
 - * Developed Python analyzer for:
 - * Event aggregation
 - * Pattern detection
 - * Attack classification
 - * Designed network-to-ML integration pipeline
 - * Enabled hybrid detection system
-

14) Significance of the Work

Without Network IDS

ML detects only data anomalies

With Network IDS

- ✓ Detects attacker behavior BEFORE impact
- ✓ Identifies reconnaissance and scanning
- ✓ Improves detection coverage

15) System Strength

- ✓ Multi-layer detection (network + data)
- ✓ Behavior-based analysis (not just signatures)
- ✓ Real-time monitoring
- ✓ Modular architecture

16) Limitations

- * Rules are broad (may generate false positives)
- * VM limits real network diversity
- * No active blocking (IDS, not IPS)

17) Future Improvements

- * Add rate-based filtering
 - * Extend rules (MQTT, CoAP)
 - * Integrate network anomaly ML
 - * Build dashboard visualization
 - * Enable auto-response (IPS)
-